LAB 08

Object Detection Using Faster R-CNN

Complete implementation, training, and held-out evaluation on a non-BCCD dataset

Course	Student details	Submission
Computer Vision (AI-4002)	Name: _____ Roll no: _____	Lab Task 6.1 Lab 08 Manual

1. Abstract and objective

This lab implements an end-to-end region-based object detector using Faster R-CNN. It follows the final requirement on page 17 of the provided Lab 08 manual: select a dataset other than Blood Cell Count and Detection (BCCD), then identify and localize objects with bounding boxes. The chosen dataset is an original, programmatically generated collection of colored geometric sign-like objects, not a public real-world dataset. Classes are circle, square and triangle.

The completed workflow generates labeled images, inspects class distributions and ground-truth boxes, creates a custom PyTorch Dataset, trains a Faster R-CNN model, validates it during training, evaluates a held-out test split, and saves both annotated images and prediction data.

2. Experiment at a glance

Property	Recorded value
Train / validation / test images	150 / 30 / 30 (210 total)
Training labeled instances	304
Test labeled instances	55
Backbone	Lightweight 4-convolution CNN (learned from scratch)
Trainable detector	Faster R-CNN: RPN + RoI Align + classification / box regression
Optimizer	Adam, learning rate 0.001, batch size 4, 12 epochs
Model selection	Highest validation mAP@0.50; checkpoint epoch 11

3. R-CNN family and algorithm

The manual distinguishes three region-based methods. Original R-CNN processes separately selected image regions, making it computationally expensive. Fast R-CNN shares feature extraction for a whole image but still relies on an external proposal mechanism. Faster R-CNN replaces external selective search with a learned Region Proposal Network (RPN), sharing image features between region generation and recognition. This implementation uses Faster R-CNN, which is explicitly covered in sections 2 and 5 of the manual.

Pipeline: RGB image (128 x 128) -> shared CNN features -> RPN candidate regions -> RoI Align (fixed-size region tensors) -> small fully connected detection head -> class scores and refined (x1, y1, x2, y2) boxes.

4. Dataset preparation and exploration

The dataset is procedurally produced with random positions, object sizes, minor background noise and faint lines. Images contain one to three objects. No BCCD image or annotation is used. The seed is 42; the generated train, validation and test examples are distinct. Annotations are stored in three CSV files with columns: filename, class, xmin, ymin, xmax, ymax. The loader converts these records into the tensor format required by TorchVision: RGB image, float32 boxes (N x 4), int64 labels, image ID, object area and iscrowd.

Split	Images	Number of objects
Train	150	304
Validation	30	63
Test	30	55

Class	Training instances
Circle	105
Square	98
Triangle	101

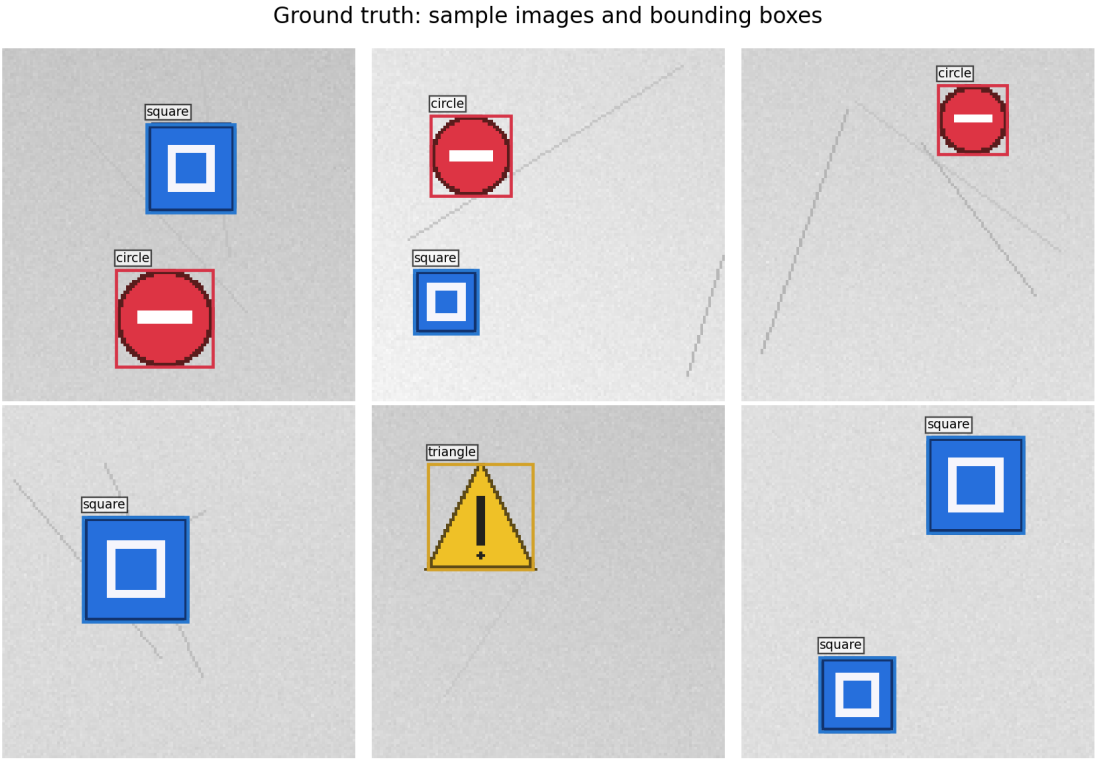


Figure 1. Generated training images with exact ground-truth bounding boxes and class labels.



Figure 2. Training object-count distribution across the three detection classes.

5. Model configuration and training

The shared CNN backbone consists of four convolutional layers with output channels 24, 48, 64 and 64, and outputs a 64-channel feature map. Its output stride is 4. The RPN uses anchor sizes 16, 24, 32, 48 and 64 pixels with aspect ratios 0.75, 1.0 and 1.33. MultiScaleRoIAlign pools proposals into 4 x 4 feature grids. A 128-unit two-layer MLP feeds classification and bounding-box regression. The classifier predicts four labels including background. The network is trained from scratch (no pretrained weights).

The training objective is the sum of the RPN objectness loss, RPN box regression loss, RoI classification loss and final box regression loss. We minimize it with Adam, learning rate 0.001 and gradient clipping at norm 5. Training uses horizontal flip augmentation (probability 0.5). Each epoch is evaluated on the validation split; the model with highest validation mAP@0.50 is retained.

Parameter	Setting
Random seed	42
Training images	150
Epochs	12
Batch size	4
Learning rate	0.001
Augmentation	Random horizontal flip on train split only
Compute	CPU, 4 threads in recorded run
Recorded training time	26.0 seconds

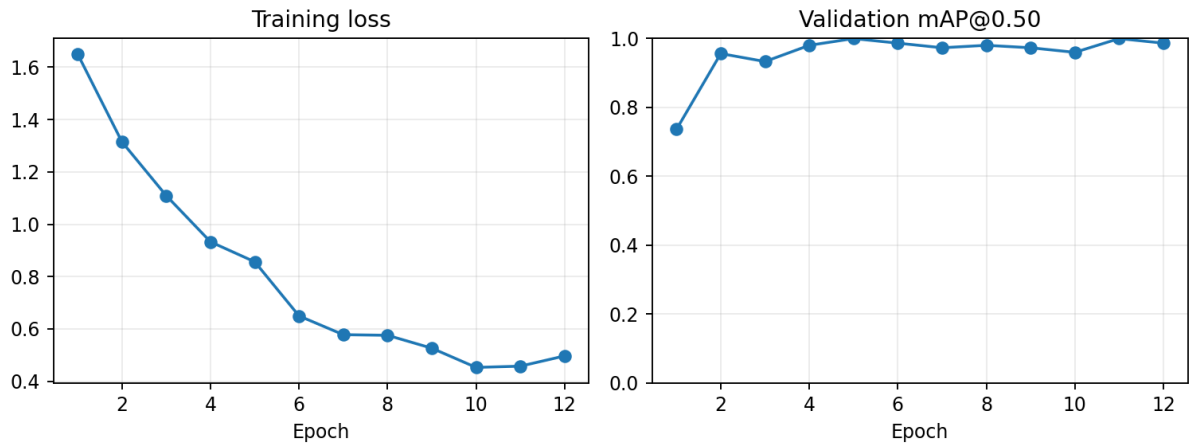


Figure 3. Optimization loss and held-out validation mAP@0.50 across training epochs.

6. Test metrics and evaluation protocol

The model is selected using validation performance only; final metrics are measured on 30 previously unseen test images. A detection counts as a true positive if it has the correct class and its Intersection over Union (IoU) with a previously unmatched ground-truth box exceeds the relevant threshold. Precision is TP/(TP+FP), recall is TP/(TP+FN), and F1 is their harmonic mean. Average precision (AP) is the area under the interpolated precision-recall curve; mAP is averaged across the three classes. Precision, recall and F1 below use confidence >= 0.35; mAP uses ranked detections without imposing that display threshold.

Metric	Actual result
mAP@IoU 0.50	1.0000 (100.0%)
mAP@IoU 0.75	1.0000 (100.0%)
Precision / Recall / F1	1.0000 / 1.0000 / 1.0000
Mean IoU of matched detections	0.9122
True positives / false positives / false negatives	55 / 0 / 0

Metric	Actual result
Test images and annotated objects	30 images / 55 objects

Class	AP50	AP75
circle	1.0000	1.0000
square	1.0000	1.0000
triangle	1.0000	1.0000

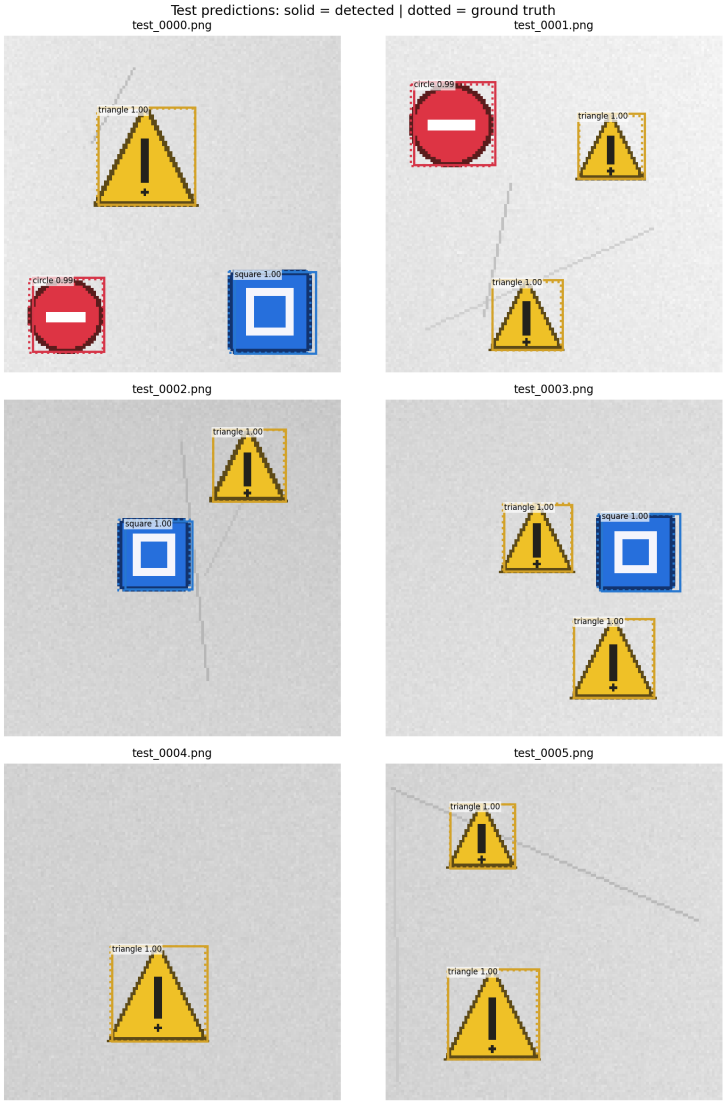


Figure 4. Inference on independent test examples. Dotted outlines show ground truth; solid outlines show predictions with confidence scores.

7. Interpretation and limitations

The model correctly identified all 55 annotated objects in the test set at IoU 0.50 and 0.75, with no false positives at confidence 0.35. Its mean matched IoU of approximately 0.912 indicates tight predicted boxes. However, this near-perfect result is expected to be easier than a real visual detection benchmark: colors are strongly correlated with labels, objects are geometrically regular, image size is fixed, backgrounds are simple and there is little overlap or occlusion. This report does not claim that performance transfers to natural traffic signs, pedestrians or other photographs.

A useful next experiment is to train the same pipeline on a separately annotated, real multi-scene dataset such as Penn-Fudan pedestrians or Pascal VOC, adding greater background diversity and evaluation at multiple IoU thresholds. Training from scratch on only 150 images is a controlled educational demonstration, not a production-ready detector.

8. Reproduction and deliverables

Inside the ZIP, run pip install -r requirements.txt once. Then run python lab08.py all --seed 42 --n-train 150 --n-val 30 --n-test 30 --epochs 12. This generates data, training plots, a trained checkpoint, test metrics and prediction images. For arbitrary image inference, run python lab08.py predict --image PATH_TO_IMAGE. The executable notebook Lab08_Faster_RCNN.ipynb mirrors all steps; saved model weights and already measured results are supplied to avoid requiring a rerun.

Delivered artifacts: complete labeled train/val/test datasets (PNG and CSV), implementation script, notebook, model checkpoint, training log and metrics (JSON/CSV), prediction CSV, images with ground-truth and predicted boxes, and this report. The generated dataset is included, so no dataset download or API key is required.

9. References

1. Lab 08 Manual, Computer Vision AI-4002, sections 2–6 (provided assignment); task statement on page 17.

2. PyTorch / TorchVision, Object Detection Finetuning Tutorial.
https://docs.pytorch.org/tutorials/intermediate/torchvision_tutorial.html

3. TorchVision documentation, Faster R-CNN and anchor generation. https://pytorch.org/vision/stable/models/faster_rcnn.html